

Mobility Diary: A Context-Aware System for Mobile Trip Acquisition, Activity Recognition, and Privacy-Aware Mobility Analysis

Giacomo Bianco

*Department of Computer Science and Engineering
University of Bologna
Bologna, Italy*

Abstract—This paper presents Mobility Diary, a context-aware mobility system designed to acquire, synchronize and visualize personal movement episodes collected through a mobile device. The system addresses the problem of battery-constrained and intermittently available mobile sensor data into a structured mobility diary that can be inspected by the user and, when needed, shared through privacy-aware representations. The proposed architecture combines on-device acquisition, finite-state tracking control, asynchronous backend ingestion, geospatial storage, Human Activity Recognition, significant-place mining, and multi-level privacy transformations.

The mobile application records GPS points, tracking state transitions, and inertial sensor windows while adapting the acquisition strategy to the current mobility context. A lightweight finite-state machine determines whether the user is stationary, potentially moving, or actively travelling, while raw sensor evidence is preserved for later enrichment. The backend receives core trip evidence through an idempotent ingestion workflow, materializes visible trips in a PostgreSQL/PostGIS database, and processes heavier raw sensor data asynchronously through Celery workers. Human Activity Recognition is performed on binary float32 sensor windows using a CNN-based feature extractor and a GRU temporal model, producing semantic mobility segments such as walking, running, cycling, vehicle movement, and idle intervals. Recurring visits are then mined from raw GPS evidence to infer user-scoped habitual places. Finally, the same diary can be exported at different privacy levels, ranging from precise personal inspection to approximate or aggregated views.

The implementation demonstrates how context-aware mobile sensing can be integrated with robust distributed-system patterns, including local persistence, retryable synchronization, object-storage uploads, asynchronous processing, geospatial querying, route assistance, personal analytics, and privacy-preserving projection. The resulting system provides a complete end-to-end pipeline from raw mobile context data to an interpretable and configurable mobility diary.

Index Terms—context-aware systems, mobile sensing, mobility diary, Human Activity Recognition, geolocation, PostGIS, asynchronous ingestion, privacy-aware data sharing, significant-place detection, Flutter, Django

I. INTRODUCTION

Modern smartphones continuously expose contextual signals that can be used to infer how people move through urban environments. GPS fixes, inertial measurements, timestamps, and application state changes provide enough information to reconstruct mobility episodes, detect stops and movements, estimate transportation modes, and identify recurring places.

However, transforming these raw signals into a reliable user-facing diary is not a trivial task. Mobile sensing is affected by battery constraints, sensor noise, operating-system interruptions, network variability, and privacy risks. A context-aware mobility system must therefore solve both an inference problem and a systems-engineering problem: it must understand the user’s mobility context while reliably transporting, processing, storing, and protecting the collected evidence.

The Mobility Diary project was developed to address this challenge. The objective is to build an end-to-end system in which a mobile application records mobility episodes, a backend platform persists and enriches them, and user-facing interfaces expose the resulting diary through maps, timelines, statistics, significant places, and privacy-aware exports. The system is not limited to raw GPS tracking. Instead, it aims to produce a semantic diary in which each trip is represented as a sequence of meaningful intervals, such as stops and movements, enriched with activity labels and recurring-place information.

A central design requirement is that data acquisition must remain feasible on a real mobile device. Continuous high-frequency tracking would provide dense evidence, but it would also consume excessive battery and produce unnecessary network traffic when the user is stationary. For this reason, the mobile application uses a finite-state acquisition model. The device distinguishes between stationary conditions, possible motion, and confirmed active tracking. In stationary conditions, sensing is reduced; when motion is suspected, the system increases the acquisition level; once a trip is confirmed, GPS points, state transitions, and inertial sensor windows are stored locally. This design reflects a core principle of context-aware systems: sensing should be adapted to the context instead of being performed uniformly at all times.

Another key requirement is reliability. A trip must not be lost if the network is temporarily unavailable, if the application is interrupted, or if the backend needs time to process heavy data. For this reason, Mobility Diary separates trip synchronization into two phases. The first phase, called core ingestion, transfers the minimum evidence needed to create a visible trip in the diary, mainly GPS points and state transitions. The second phase, called raw sensor ingestion, transfers heavier inertial sensor windows used for final enrichment. This sepa-

ration allows the user interface to return quickly to an idle state after stopping a trip, while the backend continues processing the remaining evidence asynchronously.

The backend architecture is designed around this separation. Core trip data is submitted through deterministic API requests and materialized into relational and geospatial database entities. Raw sensor windows are uploaded as compressed binary parts through an object-storage workflow and are later processed by background workers. This avoids blocking mobile interactions on expensive machine-learning inference and allows retries, idempotency, and workload isolation. The backend uses PostgreSQL with PostGIS for geospatial persistence, Redis and Celery for asynchronous execution, and Django/Django Ninja for the application API layer.

Human Activity Recognition is used as an enrichment stage rather than as the sole controller of the acquisition process. The mobile finite-state machine answers the operational question of whether the device should currently track a trip. The HAR pipeline answers a different semantic question: which activity is represented by a sensor window or a time interval. In the final processing stage, inertial windows are classified into labels such as idle, walking, running, biking, and moving vehicle. These labels are then used to construct mobility segments and provide a more readable diary. This separation prevents activity recognition errors from directly controlling trip lifecycle decisions and keeps the tracking logic robust under uncertain sensor conditions.

The system also incorporates significant-place recognition. After a trip has been enriched, the backend mines raw GPS evidence to detect candidate visits. Recurring visits are clustered per user to infer habitual places, such as home, university, work, gym, or other meaningful locations. This process is user-scoped and asynchronous, because significant places emerge from repeated mobility behavior across multiple trips rather than from a single trace. Confirmed or manually reviewed places can later be overlaid onto diary stops, making the diary more interpretable while preserving the distinction between raw GPS points, candidate visits, and stable place knowledge.

Finally, privacy is treated as an architectural concern rather than as a presentation-only feature. The same detailed diary can be projected into multiple privacy levels. A precise view preserves the full personal representation. An approximate view keeps the narrative sequence of the trip but masks exact places and cloaks coordinates. An aggregated view removes event-level paths and place identities, exposing only broader temporal and activity patterns. This design allows the system to support both personal inspection and controlled sharing without duplicating diary-generation logic.

The remainder of this report is organized according to the required project structure. Section II describes the high-level architecture, the mapping to the Privacy-Aware Mobility Diary with HAR specification, and the main mobile, backend, storage, asynchronous processing, route-assistant, analytics, and privacy layers. Section III presents the implementation details, including the Flutter acquisition logic, the ingestion API, the binary raw sensor format, the HAR pipeline, significant-place

mining, privacy-aware export rules, and dashboard functions. Section IV reports the validation strategy and results, including unit tests, integration tests, architectural boundary tests, ingestion reliability checks, privacy metrics, and functional evaluation of diary generation.

II. PROJECT'S ARCHITECTURE

A. Architectural Overview

Mobility Diary follows a layered, distributed architecture composed of a mobile acquisition client, an application backend, asynchronous workers, persistent storage, object storage, and visualization clients. The architecture is intentionally divided into components with different temporal responsibilities. The mobile client is responsible for local sensing and immediate user interaction. The backend API is responsible for authentication, ingestion orchestration, query serving, and domain rules. Celery workers are responsible for expensive or delayed computation, including final Human Activity Recognition and significant-place mining. PostgreSQL with PostGIS stores structured and geospatial domain entities, while S3-compatible object storage is used for large raw sensor payloads.

The high-level flow is the following:

```
Mobile acquisition
-> local SQLite persistence
-> core ingestion API
-> visible trip in PostGIS
-> raw sensor upload through object storage
-> Celery HAR processing
-> diary segments and significant places
-> mobile and web visualization
```

This design separates the user-perceived lifecycle from the computational lifecycle. A trip becomes visible as soon as core ingestion succeeds, while final enrichment may continue in the background. The user is therefore not forced to wait for raw sensor upload, model execution, clustering, or diary projection before seeing the trip.

B. Mapping to the Project Specification

The project is aligned with the second project proposal of the course, namely *Privacy-Aware Mobility Diary with HAR*. Table I summarizes how the main requirements of the specification are represented in the implemented system. This mapping is useful because the same platform contains several additional engineering components, such as asynchronous upload and active-trip recovery, but its core evaluation target remains the mobility-diary track.

C. Mobile Context Acquisition Layer

The mobile layer is implemented in Flutter and uses a domain-oriented structure based on repositories, Cubits, data transfer objects, and feature-specific domain models. The acquisition subsystem uses local SQLite storage through Drift and collects data from geolocation and inertial-sensor plugins. The main mobile dependencies include `flutter_bloc`, `dio`, `drift`, `sensors_plus`,

TABLE I
MAPPING BETWEEN THE COURSE SPECIFICATION AND THE IMPLEMENTED SYSTEM

Specification item	Implemented component	Role in the system
GPS trace collection	GpsPoint, mobile acquisition runtime, local Drift database	Timestamped coordinates are collected locally and later materialized in PostGIS as the spatial evidence of a trip.
Sensor data for HAR	HarSensorWindow, binary raw parts, SensorWindow compatibility model	Accelerometer and gyroscope windows are preserved as raw evidence for final activity recognition.
Mobility trace model	Trip, TripIngestion, TripIngestionPart	A user trip is separated from the upload lifecycle, allowing retries and asynchronous completion.
Mobility segments	MobilitySegment, VirtualStopInterval	The final diary is represented as ordered movement and stop intervals with timestamps, distance, geometry, and activity labels.
Activity labels	ActivityLabel, CNN feature extractor, GRU classifier	The HAR layer maps sensor windows to IDLE, WALKING, RUNNING, BIKING, and MOVING_VEHICLE.
Significant places	CandidateVisit, HabitualPlace, PostGIS DBSCAN	Recurring stay evidence is clustered per user and can be confirmed, rejected, or manually labelled.
Private and shareable diary	build_trip_privacy_export, privacy_metrics	Precise, approximate, and aggregated views are generated from the same canonical diary.
Dashboard and analytics	Vite dashboard, mobile analytics page, analytics selectors	Maps, timelines, activity distributions, heatmaps, frequent routes, and privacy comparisons are exposed to users or trusted operators.
Route assistant extension	Route-assistant Cubit, geocoding/routing services, backend classifier	A destination can be selected and the route mode can be driven by manual choice or live sensor-based classification.

geolocator, mapbox_maps_flutter, flutter_secure_storage, crypto, and latlong2.

The acquisition logic distinguishes between stationary and movement states at the domain level and exposes them to the user interface as stationary, potential-motion, and active-tracking conditions. These states determine the sensing strategy. When the user is stationary, the application minimizes sensing. When possible movement is detected, additional evidence is collected to confirm whether a trip is starting. When active tracking is confirmed, GPS points, state transitions, and HAR sensor windows are persisted locally. The local-first design makes the mobile application resilient to temporary network failures and allows synchronization to be retried after the recording has ended.

The finite-state machine uses both inertial and spatial context. Motion evidence is accepted only when a fresh accelerometer-derived sigma value and a fresh GPS speed value jointly support movement. In the implemented configuration, movement requires at least 15 seconds of evidence, a GPS speed of at least 0.8 m/s, and a sigma value of at least 1.2. Conversely, a return to stationary status requires a longer stationary evidence interval, set to 120 seconds, with GPS speed below 0.4 m/s and sigma below 0.8. This asymmetry avoids frequent oscillations and reflects the fact that stopping a trip should require more confidence than starting one.

D. Asynchronous Ingestion Architecture

The ingestion architecture is based on two separate phases: core ingestion and raw sensor ingestion. Core ingestion is the primary path for creating a diary-visible trip. It contains the minimum evidence required to reconstruct the route and the state-transition history. Raw sensor ingestion transfers

high-volume inertial windows used by the backend for final enrichment.

Core ingestion is submitted through deterministic JSON payloads. The backend computes and verifies stable hashes in order to support idempotent retries. Raw sensor ingestion is instead uploaded as compressed binary parts. The backend does not receive large raw bytes directly through the application process. It issues presigned URLs, the mobile client uploads parts to object storage, and the backend confirms the existence and integrity of each part. After all raw parts have been confirmed, a final HAR job is queued.

This architecture provides several benefits. First, it prevents large mobile uploads from occupying web workers. Second, it allows the mobile client to retry individual parts instead of resending an entire trip package. Third, it decouples upload completion from CPU-intensive processing. Fourth, it scales naturally from single-device demonstrations to a larger number of concurrent users because expensive work is moved to worker queues.

The ingestion state machine is explicitly represented in the database. Core and raw phases can independently be pending, receiving, received, queued, processing, completed, retryable failure, or final failure. The mobile application interprets these states differently: `core_status=COMPLETED` means that the trip can be displayed, whereas `raw_status=COMPLETED` means that the diary has been fully enriched. This distinction is important for the user experience because the trip list does not depend on the completion time of model inference.

E. Backend Domain and Service Layer

The backend is implemented with Django and Django Ninja. Django provides the relational model, migrations, authenti-

cation integration, and administrative interface, while Django Ninja exposes typed HTTP APIs. The backend codebase uses a service-layer architecture in which API routers remain thin and delegate domain operations to services and selectors. This avoids placing business logic directly inside controllers or ORM models and makes the ingestion, trip, privacy, analytics, and place-mining workflows easier to test.

The main domain objects include users, trips, GPS points, state transitions, trip ingestions, ingestion parts, HAR jobs, mobility segments, virtual stop intervals, candidate visits, and habitual places. The relation between the entities follows the terminology requested by the project specification. A `Trip` corresponds to the mobility trace; `GpsPoint` stores timestamped spatial samples; `MobilitySegment` represents the diary intervals; `ActivityLabel` represents the recognized mobility mode; and `HabitualPlace` represents the significant-place abstraction. PostGIS is used for geospatial fields such as GPS points, trip paths, mobility-segment paths, visit centers, and significant-place centers. Redis is used as the Celery broker and supports background processing.

The API surface follows this same model. Core mobile APIs allow the client to create or synchronize trips, upload GPS points, upload state transitions, submit sensor windows, close trips, fetch tracks, fetch diary segments, retrieve significant places, and request privacy-aware exports. Web APIs expose staff-oriented dashboards, user trip lists, private and privacy-aware tracks, and privacy metrics. The backend therefore supports both the mobile owner-facing scenario and the web inspection scenario required for demonstration.

F. Human Activity Recognition and Diary Enrichment

The HAR layer processes raw inertial windows after a trip has been synchronized. The mobile app stores each standard window as a float32 matrix with 500 samples and 6 channels: accelerometer x, y, z and gyroscope x, y, z. The backend accepts the current binary format and remains compatible with older JSON-based payloads. The final pipeline loads raw windows, normalizes them, applies a CNN feature extractor and a GRU temporal model, corrects idle predictions using GPS speed when appropriate, and writes mobility segments.

This deferred enrichment strategy is important because the GRU model benefits from temporal context. Instead of forcing the classifier to make immediate isolated decisions during live acquisition, the final backend job can process the entire trip with a broader view of surrounding windows. The result is a diary timeline that is semantically richer than a raw GPS trace.

G. Route Assistant Architecture

The route assistant is an optional context-aware extension connected to the same mobility-mode vocabulary used by HAR. The user can select a destination through a search sheet. The client determines the origin from the active recording location when a trip is in progress, or from the current GPS location otherwise. A route is then requested for a selected mode, such as walking, cycling, or driving.

The assistant can also operate in a live mode. In this mode, the mobile client periodically obtains a current sensor window, sends it to the backend classifier, maps the recognized activity to a navigation mode, and refreshes the route. The mapping is intentionally simple and explainable: walking and running map to a pedestrian route, biking maps to a cycling route, and moving vehicle maps to a driving route. Thus, the route assistant uses context in two ways: current location determines the route origin, and recognized mobility mode determines the route profile.

H. Personal Mobility Analytics

The analytics layer aggregates the diary across multiple trips. Daily and weekly buckets are computed from mobility segments and are grouped into categories such as stopped time, walking, running, cycling, and vehicle movement. Additional analytics include the prevalent mobility mode, frequent routes between known habitual places, and heatmap points weighted by significant-place visit counts. These analytics are exposed through mobile pages and web dashboards, satisfying the requirement that the diary should provide personal statistics rather than only raw trace visualization.

I. Significant-Place Mining

Significant-place recognition is implemented as a user-scoped asynchronous backend step. The algorithm starts from raw GPS evidence, detects candidate visits based on temporal permanence inside a spatial radius, and clusters recurring visits into habitual places. The process deliberately uses raw GPS points rather than already projected diary stops as the source of truth. This avoids circular dependencies between the diary representation and place discovery.

The clustering phase uses PostGIS DBSCAN over projected visit centroids when PostgreSQL/PostGIS is available. A Python fallback exists for non-PostgreSQL test environments. Manual review is preserved across recomputations, so user-confirmed places are not lost when the backend recomputes candidate visits from the full history. A PostgreSQL advisory lock serializes mining per user, preventing concurrent trips from producing conflicting place updates.

J. Privacy-Aware Diary Architecture

Privacy is implemented through centralized diary projection. The detailed private diary is first constructed from canonical domain entities such as mobility segments, virtual stop intervals, GPS points, confirmed habitual places, and HAR labels. Privacy levels then transform the publishable representation rather than reconstructing separate diaries independently.

Three levels are supported. The precise level preserves the full owner-facing diary. The approximate level preserves the sequence of stops and movements but rounds time intervals, cloaks movement paths, masks place names, and removes precise stop centers. The aggregated level removes event-level paths and places, grouping mobility information by broad time bands and publishing only activity patterns, approximate distance buckets, and total durations. This design allows the

same trip to support both personal inspection and controlled sharing.

III. PROJECT'S IMPLEMENTATION

A. Mobile Implementation

The mobile application is implemented in Flutter. The project uses a modular structure that separates domain entities, repositories, network services, state-management Cubits, mappers, and UI pages. Acquisition-related domain models include route points, tracking events, sampling profiles, motion metrics, sensor windows, synchronization snapshots, and finite-state-machine state. This separation keeps UI concerns independent from synchronization and sensing logic.

During a trip, evidence is persisted locally before it is sent to the backend. This is essential because mobile connectivity is not guaranteed. GPS points and state transitions form the core trip evidence. HAR windows form raw evidence for final enrichment. The local database stores the matrix of each standard sensor window as a binary blob, avoiding the overhead of JSON serialization for high-frequency sensor matrices. A standard production window contains 500 samples, 6 channels, and 4 bytes per float, resulting in 12,000 bytes before compression.

When the user stops a trip, the mobile application closes the local recording session and creates or updates a persistent synchronization job. The UI can return to an idle state immediately. The synchronization queue then builds a deterministic core JSON payload, submits it to the backend, receives the corresponding ingestion and trip identifiers, uploads raw binary parts with bounded parallelism, and polls the backend while the server-side enrichment is queued or running.

B. Adaptive Sampling and FSM Implementation

The acquisition finite-state machine receives synthesized events rather than raw sensor streams. The main event types are motion-window evaluations, GPS fixes, and timeout events. This makes the FSM independent from the concrete sensor runtime and easier to test. The decision output contains the next tracking state, the sampling profile to be applied, and an optional transition record. Each transition stores its source state, target state, timestamp, and reason, and can later be synchronized to the backend as part of the core evidence.

The sampling profile is the bridge between context interpretation and device behavior. In stationary conditions, the profile uses lower-frequency sensing and avoids HAR window persistence. During confirmed tracking, GPS points and HAR windows are persisted. The system therefore adapts resource consumption to the inferred context. This is a core design choice for a mobile context-aware system, because collecting the maximum amount of data continuously would be simpler but would not be appropriate for battery-constrained usage.

The synchronization queue is also implemented as a context-aware background component. It processes only authenticated jobs, performs packaging, uploads the core payload first, uploads missing raw parts later, applies retry and backoff policies, and avoids duplicate remote trips through backend

idempotency. Raw upload concurrency is bounded to avoid uncontrolled network pressure from the mobile device.

C. Backend Ingestion API

The ingestion API is implemented in the Django Ninja application. The core path uses `POST /api/ingestion/trips/core`. This endpoint validates the canonical hash of the payload, creates or retrieves the corresponding `TripIngestion`, materializes the `Trip`, `GpsPoint`, `StateTransition`, and route geometry records, and marks the core phase as completed. Once the core phase is complete, the trip is visible to the user even if raw enrichment is still pending.

Raw sensor ingestion follows a presign-confirm-complete protocol. The mobile client asks the backend for a presigned URL for a raw part, uploads the compressed binary object directly to the storage service, and then confirms the part. Confirmation checks object existence, size, and SHA-256 metadata. When all expected parts have been confirmed, the client calls the raw completion endpoint. The backend marks the raw phase as queued and creates a final HAR job for Celery.

Table II summarizes the most relevant API families. The same backend exposes additional endpoints for authentication, privacy settings, web-dashboard users, and staff-oriented inspection.

TABLE II
REPRESENTATIVE API FAMILIES

API family	Purpose
Core ingestion	Submit deterministic core evidence and create a visible trip.
Part presign	Obtain signed upload URLs for part-based transfer.
Part confirm	Confirm object-storage part availability and integrity.
Trip diary	Retrieve the owner-facing diary timeline.
Privacy export	Retrieve a privacy-aware textual/mobile export.
Place review	Review, confirm, reject, reactivate, or label significant places.
Analytics	Retrieve personal mobility analytics.
Route assistant	Classify a sensor window into a route mode.

The raw object key is deterministic. A representative simplified function is shown in Listing 1. The function intentionally contains no inline explanatory comments, in accordance with the code-comment policy.

Listing 1. Deterministic object key construction for ingestion parts.

```
def _object_key(base_path: str, kind: str, sequence:
    int) -> str:
    if kind == PartKind.SENSOR_WINDOWS:
        return f"{base_path}sensor_windows_part_{
            sequence:04d}.bin.gz"
    return f"{base_path}{kind}.json.gz"
```

D. Raw Sensor Binary Format

The current raw sensor format is a compressed binary file named according to the pattern `sensor_windows_part_XXXX.bin.gz`. After decompression, the payload is little-endian and starts with the magic value `MDHARW1`. The global header contains the number of windows. Each window stores start and end timestamps in UTC microseconds, sampling frequency, sample count, channel count, and the contiguous float32 sample matrix.

This format replaced a previous JSON-based representation. The binary representation reduces payload size, avoids expensive JSON parsing for dense matrices, and maps naturally to NumPy arrays in the backend. Compatibility with legacy JSON payloads is preserved where needed, but the production path uses binary float32 windows with 6 channels.

E. HAR Processing Task

Final HAR processing is executed by a Celery task. The task claims the raw ingestion idempotently, loads raw parts from object storage, decompresses them, decodes binary windows, sorts them, normalizes the sensor matrices, executes the model pipeline, applies GPS-based corrections, writes mobility segments, marks the trip as processed, and requests significant-place mining for the trip owner.

The implemented model pipeline uses a two-stage architecture. A 1D convolutional neural network receives single inertial windows and acts as a feature extractor. The extracted embeddings are then processed by a GRU model over sequences of 32 windows, allowing the classifier to exploit temporal context. The five output classes are `IDLE`, `WALKING`, `RUNNING`, `BIKING`, and `MOVING_VEHICLE`. The current production path uses 6 sensor channels: accelerometer `x`, `y`, `z` and gyroscope `x`, `y`, `z`. Earlier design notes considered 9 channels, including magnetometer samples; the final implementation removed the magnetometer from the HAR path and retained compatibility code for older payloads.

The worker records detailed timing fields, including raw storage read time, gzip decompression time, binary decode time, normalization time, classification time, GPS correction time, segment database write time, and total pipeline time. These timings are useful for diagnosing performance bottlenecks and for demonstrating that the system is not a monolithic synchronous endpoint.

The HAR processing flow is summarized in Listing 2.

Listing 2. Final HAR processing flow.

```
load confirmed raw parts
decode MDHARW1 binary windows
normalize accelerometer and gyroscope channels
run CNN feature extraction and GRU classification
correct idle intervals using GPS speed
write mobility segments and virtual stops
mark trip as processed
queue significant-place mining
```

Ambiguous mobility cases are addressed through the separation between tracking control and diary enrichment. An

`IDLE` prediction does not automatically close a trip, because it may correspond to traffic lights, congestion, a train stop, or a temporary pause. Similarly, walking and running are both interpreted as human movement, while the route assistant maps both to a pedestrian navigation profile. GPS speed correction is applied after neural classification to reduce inconsistent idle intervals when the spatial evidence indicates movement.

F. HAR Evaluation Protocol

The project includes the trained Keras models and the backend/mobile integration needed to run inference over real or replayed sensor windows. For a complete experimental evaluation, the HAR model should be assessed on a labelled hold-out set using the metrics requested by the project specification: accuracy, precision, recall, F1-score, and confusion matrix. The expected evaluation protocol is the following: split labelled windows by recording session to avoid leakage across adjacent windows, run the CNN extractor and GRU classifier on the test sequences, compare predicted classes against ground truth labels, and report per-class metrics for the five activities.

The implemented codebase already validates the integration boundaries around HAR, including invalid-window rejection, mapping from HAR labels to navigation modes, raw binary decoding, final pipeline orchestration, and diary segment generation. These tests do not replace a dataset-level accuracy benchmark, but they verify that the application pipeline can consume HAR predictions correctly. The final report should be updated with numerical HAR metrics when the labelled evaluation run is executed.

G. Significant-Place Implementation

The significant-place module is divided into visit detection and place mining. Visit detection scans ordered GPS points and maintains a candidate permanence cluster. A cluster is flushed as a visit only if it contains at least three valid points and lasts at least five minutes inside a 75-meter radius. GPS points with accuracy worse than 100 meters are ignored. A sampling gap longer than the minimum stay threshold closes the current permanence candidate.

After visits are detected, the mining phase deletes and re-generates non-manual candidate evidence for the user, clusters visits spatially, creates or reconnects habitual places, and links candidate visits to their place. A cluster becomes a stronger place hypothesis when it is observed across multiple days. Places can also be manually reviewed by the user, and manual labels take precedence over automatically inferred neutral labels.

H. Privacy-Aware Export Implementation

Privacy-aware export is centralized in the diary export module. The builder receives a trip and a privacy level, projects the detailed diary, and then applies the selected transformation. For approximate exports, timestamps are rounded to five-minute boundaries, paths are spatially cloaked, real place names are replaced with category-level labels, and movement distance

is recomputed on the cloaked path. For aggregated exports, the event-level sequence is compressed into broad temporal bands such as night, morning, afternoon, and evening. Path coordinates and precise places are omitted entirely.

The main benefit of this implementation is consistency. The mobile privacy export endpoint and the web dashboard use the same builder for the diary narrative. Therefore, privacy rules are not duplicated across presentation layers, and a change in the policy can be tested in one place.

I. Privacy Metrics Implementation

The privacy-aware dashboard computes two families of metrics. The first is Privacy Perturbation, defined as the distance between each original trajectory point and its published cell center. The system reports the mean perturbation, maximum perturbation, and number of evaluated samples. The second is Quality of Service, defined as the relative error between the private route distance and the privacy-aware route distance:

$$QoS_{loss} = \frac{|d_{private} - d_{published}|}{d_{private}}. \quad (1)$$

For the approximate level, the spatial cloaking grid uses 150-meter cells. For the aggregated level, the grid uses 400-meter cells and the diary removes event-level paths and precise places. The web dashboard exposes these metrics as comparison cards and allows previewing a different privacy level without changing the user's saved default preference. This directly supports the project requirement to compare privacy perturbation and quality loss.

J. Route Assistant Implementation

The route assistant is implemented on the mobile side as a dedicated Cubit, independent from the acquisition Cubit. This separation is important because route assistance is an interactive feature, while acquisition is a recording lifecycle. The user selects a destination through a search interface; the route origin is derived from the active trip location when available or from the current GPS location otherwise. The selected or detected route mode is then used to request a walking, cycling, or driving route.

In live mode, the assistant periodically classifies the current sensor window through the backend route-assistant classifier. The backend validates the window shape and calls the HAR adapter. The returned activity is mapped to a navigation mode: WALKING and RUNNING become walking, BIKING becomes cycling, and MOVING_VEHICLE becomes driving. This implements the route-assistant extension requested by the specification, where route suggestions react to the user's context.

K. Personal Analytics Implementation

Personal analytics are computed from persisted mobility segments and habitual places. The backend groups activity duration and distance by daily or weekly buckets, maps activity labels to user-facing mobility categories, computes the

prevalent non-idle mode, derives heatmap points from confirmed habitual places, and extracts frequent routes between known places. The mobile analytics page visualizes time by activity and frequently visited places, while the web dashboard provides inspection views for trips, timelines, maps, private tracks, privacy-aware tracks, and metric comparisons.

L. Web Dashboard and Authentication

In addition to the mobile client, the project includes a Vite-based web platform. The web interface supports authenticated access, trip dashboards, privacy dashboards, user lists for trusted operators, maps, timelines, analytics views, and inspection of privacy-aware diary projections. Authentication is implemented through backend account APIs, token handling, and session persistence. The web layer consumes the same backend APIs used by the mobile client where appropriate, while adding staff-oriented global views for demonstration and analysis.

M. AI Usage Disclosure

The project rules explicitly require transparency regarding AI-tool usage and restrict comments in source code to the beginning of files and to the beginning of functions or methods. The implemented codebase follows this convention in the reviewed modules: explanatory comments are placed in module headers or function-level docstrings, while significant inline commentary is avoided.

During the preparation of this technical report, AI-based assistance was used only as a documentation-support tool. In particular, OpenAI Codex was used to help organize the report structure, reformulate technical explanations in academic English, and align the written description with the implemented architecture. The submitted source code was not generated to a significant extent by AI tools, and all implementation choices, APIs, data models, and programming constructs were reviewed and understood by the author. This disclosure is included in accordance with the project AI policy.

IV. RESULTS

A. Functional Results

The implemented system provides a complete path from mobile sensing to diary visualization. A user can record a trip on the Flutter application, store the evidence locally, stop the trip without waiting for heavy backend processing, synchronize the core trip evidence, upload raw sensor evidence, and later inspect the enriched diary. The backend materializes route geometry, GPS evidence, state transitions, mobility segments, significant places, and privacy-aware exports. The mobile and web clients can then display trip lists, trip details, maps, timeline entries, analytics, place information, and privacy-aware diary views.

The main qualitative result is the successful separation between visibility and enrichment. A trip can become visible after core ingestion, while raw HAR processing and significant-place mining continue asynchronously. This behavior is appropriate for a context-aware mobile system because

it reduces perceived latency and prevents the user interface from depending on machine-learning processing time.

Table III summarizes the coverage of the optional components described in the Privacy-Aware Mobility Diary proposal. The strongest implemented extensions are advanced significant-place recognition, privacy-aware diary generation with metrics, personal analytics, route assistance, and Docker-based deployment.

TABLE III
COVERAGE OF OPTIONAL PROJECT COMPONENTS

Component	Implemented coverage
Advanced HAR	CNN+GRU inference pipeline over inertial windows; integration tests and diary use.
Route assistant	Destination search, route mode selection, live HAR-based mode adaptation.
Advanced significant places	Stay detection, PostGIS DBSCAN clustering, manual labels, diary overlay.
Location privacy	Precise, approximate, and aggregated views with cloaking and QoS metrics.
Personal analytics	Time by activity, heatmaps, prevalent mode, frequent routes.
Docker Compose	Backend, database, Redis, worker, gateway, and deployment compose files.

B. Reliability Evaluation

Reliability was evaluated through the ingestion design and through automated tests. The mobile application stores evidence locally before synchronization, so temporary connection failures do not immediately imply data loss. The backend ingestion workflow is idempotent, uses stable hashes for core payloads, and separates part upload from part confirmation. Raw evidence is divided into parts, allowing partial retries. The backend state model distinguishes pending, receiving, received, queued, processing, completed, retryable failure, and final failure states.

The architecture also handles active-trip consistency. The backend maintains the concept of a single active recording per trip owner and associates the recording with an origin device. This prevents two devices from recording conflicting active trips for the same account. Heartbeats and abandonment rules allow stale recordings to be released after a defined period or when the origin device reports that it can no longer resume the local session.

C. Testing Strategy

The repository contains backend and mobile tests covering the most important domain boundaries. Backend tests verify ingestion APIs, ingestion services, raw sensor codec behavior, trip services, trip selectors, place services, route-assistant services, reload services, analytics selectors, diary export behavior, and architecture boundaries. Account tests verify registration, web authentication, web user APIs, and

privacy settings APIs. Mobile tests cover Cubits, repositories, DTOs, Drift database behavior, acquisition runtime components, synchronization queue behavior, FSM behavior, motion metrics, HAR sensor windows, UI widgets, and architecture import boundaries.

Table IV summarizes representative tested areas.

TABLE IV
REPRESENTATIVE VALIDATION AREAS

Area	Validation focus
Ingestion API	Core submission, raw part lifecycle, status transitions, idempotent behavior
Raw sensor codec	Binary payload parsing, legacy compatibility, invalid payload handling
HAR pipeline	Final processing orchestration, segment generation, timing instrumentation
Significant places	Visit detection, clustering, manual review preservation, user scoping
Privacy export	Precise, approximate, and aggregated diary transformations
Mobile acquisition	FSM behavior, sensor windows, local database, synchronization queue
Architecture boundaries	Import rules and separation between service, selector, API, and UI layers

D. Context-Awareness Evaluation

The project satisfies the main properties expected from a context-aware system. It senses environmental and user-related context through GPS and inertial sensors. It interprets context through an acquisition finite-state machine, HAR classification, stop detection, and significant-place mining. It adapts behavior to context by changing sensing intensity across stationary, potential-motion, and active-tracking states. It produces user-facing context representations through trip timelines, mobility categories, place labels, and analytics. It also adapts disclosure according to privacy context by transforming the diary into precise, approximate, or aggregated forms.

The architectural distinction between operational context and semantic context proved especially important. The FSM uses simple, robust signals to decide whether tracking should be active, while HAR assigns semantic activity labels after sufficient evidence has been collected. This prevents the system from prematurely closing a trip because of an isolated idle prediction and allows richer temporal models to operate after the full trip is available.

E. Performance-Oriented Design Outcomes

Although the final evaluation was primarily functional, several implementation choices directly improve performance. The binary raw sensor format removes JSON overhead for high-frequency matrices. Gzip compression reduces network and storage volume. Direct object-storage uploads prevent the Django web process from handling large raw bytes. Celery workers isolate CPU-intensive model execution from request-response APIs. PostGIS performs spatial operations inside the

database, including route geometry handling and DBSCAN-based clustering. Local SQLite persistence allows the mobile client to batch and retry synchronization rather than issuing a fragile stream of tiny network requests.

The system is therefore designed to scale beyond a single demonstration device. The intended deployment can run multiple web replicas behind a gateway, multiple Celery workers consuming the queue in parallel, PostgreSQL/PostGIS for structured geospatial persistence, Redis for task coordination, and S3-compatible storage for raw evidence. This does not remove the need for production monitoring, but it establishes a clear path from prototype to a more robust service.

F. Summary

The final system demonstrates a complete context-aware mobility pipeline. It records context on a mobile device, synchronizes it reliably, enriches it asynchronously, mines recurring places, and exposes the result through privacy-aware views. The strongest result is the integration of mobile sensing, distributed ingestion, geospatial storage, machine-learning enrichment, and privacy projection into a coherent architecture. This combination makes Mobility Diary suitable as a course project for Context-Aware Systems because it addresses sensing, interpretation, adaptation, and user-facing contextual services within a single implemented platform.

ACKNOWLEDGMENT

This project was developed for the Context-Aware Systems course at the University of Bologna, held by Prof. Marco Di Felice and Prof. Angelo Trotta.

REFERENCES

- [1] Flutter documentation, “Build apps for any screen,” Google. [Online]. Available: <https://docs.flutter.dev/>
- [2] Django Software Foundation, “Django documentation.” [Online]. Available: <https://docs.djangoproject.com/>
- [3] PostGIS Project Steering Committee, “PostGIS documentation.” [Online]. Available: <https://postgis.net/documentation/>
- [4] Celery Project, “Celery distributed task queue documentation.” [Online]. Available: <https://docs.celeryq.dev/>
- [5] IEEE, “IEEE conference templates.” [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>